



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Inteligência Artificial para Sistemas Autónomos
Relatório: Projeto (Entrega 1)

Turma: 42D

Nome: Miguel Alcobia

Número: 50746

Professor: Luís Morgado

Data: 27 de Abril de 2024

Índice

Lista de Tabelas	III
Introdução	1
Enquadramento Teórico	2
Inteligência Artificial	2
Engenharia de Software	3
Projeto - Parte 1	5
Projeto - Parte 2	9
Revisão do projeto realizado	17
Conclusão	18
Bibliografia	19

Lista de Figuras

Figura 1 - Miles, R., & Hamilton, K. (2006) Escala de dependência	3
Figura 2 - Morgado, Luís. (2024). Modelo de dinâmica de um sistema	4
Figura 3 - Morgado, Luís. (2024). Representação conceptual da relação entre agente e ambiente	5
Figura 4 - Morgado, Luís. (2024). Dinâmica da Personagem	8
Figura 5 - Demonstração do jogo feito na Parte 1	9
Figura 6 - Morgado, Luís. (2024). Arquitetura de um agente reativo	9
Figura 7 - Morgado, Luís. (2024). Associação Estímulo-Resposta	10
Figura 8 - Morgado, Luís. (2024). Esquema da associação Estímulo-Resposta	10
Figura 9 - Morgado, Luís. (2024). Comportamentos Compostos	11
Figura 10 - Morgado, Luís. (2024). Seleção por Hierarquia	11
Figura 11 - Morgado, Luís. (2024). Seleção por Prioridade	11
Figura 12 - Morgado, Luís. (2024). Seleção por Combinação	12
Figura 13 - Morgado, Luís. (2024). Agente com Controlo Reactivo	12
Figura 14 - Morgado, Luís. (2024). Comportamento AproximarAlvo	14
Figura 15 - Morgado, Luís. (2024). Resposta a Estímulos	15
Figura 16 - Morgado, Luís. (2024) - Comportamento Recolher	16
Figura 17 - Apresentação do resultado do projeto no final da Parte 2	16

Lista de Tabelas

Tabela 1 - Tabela com os valores do dicionário de AmbienteJogo 6

Introdução

Este trabalho foi realizado na âmbito da Unidade Curricular de Inteligência Artificial para Sistemas Autónomos. A disciplina tem como objetivo fazer com que o aluno adquira e desenvolva conhecimentos nas áreas de Inteligência Artificial e Engenharia de Software, durante o semestre.

O estudo sobre Inteligência Artificial incide sobre o desenvolvimento e compreensão de vários tipos de agentes e dos diversos mecanismos que dão suporte ao seu funcionamento. Já o estudo em Engenharia de Software aborda diversas técnicas e boas práticas para que o programador consiga construir sistemas complexos de forma atempada e com uma alta qualidade.

O objetivo deste relatório é detalhar o processo de desenvolvimento do projeto que é, nestas primeiras duas partes, criar um agente reativo que se desloque por um mapa, enquanto evita obstáculos e recolhe alvos. Também serão justificadas as decisões tomadas com base na teoria estudada durante as aulas.

O desenvolvimento do projeto está dividido em quatro partes a serem realizadas ao longo do semestre.

Enquadramento Teórico

Inteligência Artificial

O campo da inteligência artificial (IA) procura entender e construir entidades inteligentes, ou seja, que podem calcular como agir de forma eficaz e segura em uma ampla variedade de situações novas, tornando-as assim capazes de comportamento inteligente.

Temos dois pontos de vista que podem ajudar a olhar para a inteligência artificial:

- **Analítico:** Baseia-se na realização de experiências que levavam à elaboração de modelos e teorias para explicar os fenómenos observados e reproduzir esses mesmos fenómenos em sistemas artificiais.
- **Sintética:** Baseia-se nesses modelos e teorias para desenvolver sistemas que apresentam características e comportamentos associados ao conceito de inteligência.

Dentro da inteligência artificial temos alguns paradigmas que merecem ser apresentados:

- **Simbólico:** Através de símbolos é processada e representada a informação e conceitos relacionados a um determinado problema.
- **Conexionista:** A informação é processada por redes interligadas de unidades de processamento elementares (neurónios) e fica nas ligações entre neurónios.
- **Comportamental:** A informação é processada através de relações entre estímulos e respostas, modeladas sob a forma de reações e de comportamentos (conjuntos estruturados de reações).

Um sistema autónomo inteligente opera por conta própria, sem precisar de outros sistemas. Este sistema trabalha segundo um ciclo de PERCEÇÃO - PROCESSAMENTO - AÇÃO e através dele controlasse o funcionamento do sistema para que seja atingida a sua finalidade.

O objetivo base do trabalho é implementar um agente que executa o ciclo acima descrito, cujo o esquema está presente na Figura 3.

Para um agente ser inteligente precisa de apresentar características fundamentais, tais como:

- **Autonomia:** Capacidade de um sistema operar de forma independente.
- **Reatividade:** Capacidade de reagir a estímulos do ambiente.
- **Pró-Atividade:** Capacidade do sistema, consoante o seu objetivo, tomar a iniciativa de ação.
- **Sociabilidade:** Capacidade de interatuar e agir com outros sistema para alcançar determinados objetivos, sejam eles comuns ou individuais.

Dentro da arquiteturas de agente podemos ter os seguintes modelo:

- **Reativo:** Tem por base um paradigma comportamental, onde o comportamento do sistema é gerado de forma reativa, com base em associações entre estímulos e respostas.
- **Deliberativo:** Tem por base um paradigma simbólico, onde o comportamento é gerado à base de mecanismos de deliberação (raciocínio e tomada de decisão), recorrendo a representações internas que

incluem objetivos explícitos.

- **Híbrido:** O comportamento é gerado através de processos que integram vertentes reativas e deliberativas.

Os mecanismos que fundamentam a implementação do agente reativo do projeto serão abordados na [Parte 2](#) do relatório à medida que forem realizados.

Engenharia de Software

Para conseguir criar sistemas complexos de forma atempada e com uma alta qualidade, tal como foi referido na [Introdução](#), recorreremos à Engenharia de Software. O grande pilar das técnicas de engenharia de software é a modelação de sistemas e, consequentemente, o uso de linguagens de modelação como a linguagem UML que esquematiza a estrutura do sistema a ser desenvolvido.

Ao recorrer às boas práticas de engenharia de software, reduzimos bastante dois grandes problemas que inevitável surgem no desenvolver de um projeto: Complexidade e Mudança. A complexidade é expressa na crescente dificuldade de desenvolvimento, operação e manutenção, bem como na quantidade de recursos envolvidos. A mudança é expressa no ritmo crescente a que o projeto precisa ser produzido e modificado, seja para adicionar algum mecanismo ou atualizar um existente.

Caso estes problemas não sejam devidamente controlados, podemos acabar por nos perder no desenvolvimento do projeto ou perder bastante eficiência da qual se podia tirar proveito.

Para lidar com a complexidade, recorreremos à arquitetura de software que tem por princípios a Abstração e a Modularização. A abstração foca-se em realçar o que é essencial para a descrição do sistema, podendo assim criar modelos abstratos e genéricos, até certo ponto, para representar o sistema, Estes modelos podem ser desenhados em diagramas, seguindo a linguagem UML.

Um conceito bastante importante a reter é o de Interface. Uma interface é uma coleção de operações que não têm implementações de métodos correspondentes [Pressman, 2010], ou seja, definem um contrato com a classe específica, permitindo a esta fazer a sua própria implementação dos métodos que possui. Desta forma, uma interface contribui para o aumento do encapsulamento da implementação.

Outras medidas importantes na arquitetura de software são:

- **Acoplamento:** Grau de dependência entre diferentes partes de um sistema, aumentando conforme aumenta a dependência entre essas partes.

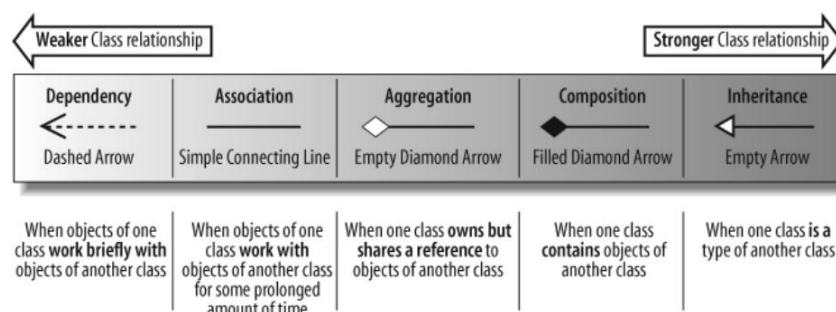


Figura 1 - Miles, R., & Hamilton, K. (2006) Escala de dependência

- **Coesão:** Forma de como os elementos de um sistema estão agrupados de forma coerente entre si, aumentando conforme as relações existentes entre elementos agrupados em cada módulo.
- **Simplicidade:** Nível de facilidade de compreender e apresentar a arquitetura do sistema.
- **Adaptabilidade:** Nível de facilidade para permitir a alteração e adição de novos requisitos ou daqueles já existentes.
- **Modularização:** Organização de um sistema em módulos que podem interligados entre si. Pode ser dividida em:
 - **Decomposição:** Decompõe um sistema em partes coesas. Podemos destacar a Fatorização que elimina redundância existente e garante consistência.
 - **Encapsulamento:** Isolamento dos detalhes internos das partes de um sistema em relação ao exterior. Desta forma reduzimos dependências existentes com a implementação de Interfaces que estabelecem um contrato funcional com o exterior.

A dinâmica do sistema é composta pelos vários estados que um sistema pode assumir e forma como eles evoluem ao longo do tempo para produzir o comportamento do sistema. Este comportamento atua as suas saídas em função das suas entradas e do seu estado interno.

Assim sendo, podemos caracterizar um sistema computacional através de uma representação de estado, que define em cada momento a configuração interna do sistema, e de uma função de transformação que gera saídas e o próximo estado em função das entradas e do estado atual do sistema.

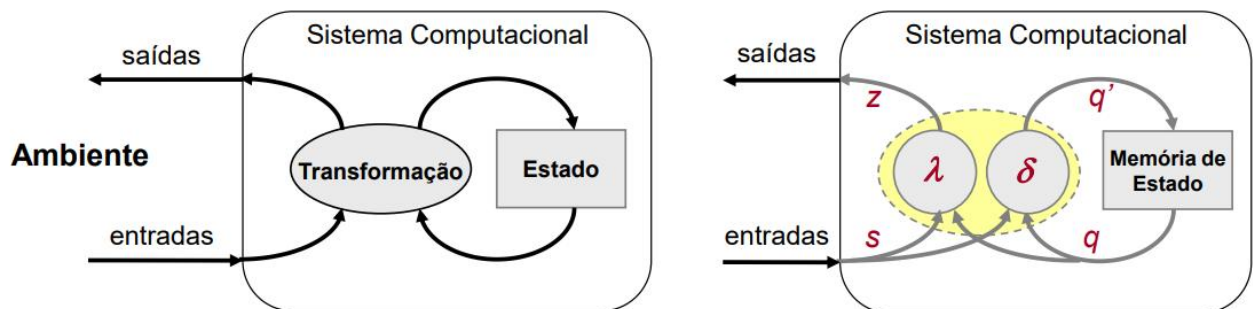


Figura 2 - Morgado, Luís. (2024). Modelo de dinâmica de um sistema

As entradas e saídas são abstraídas em alfabetos (Σ é o alfabeto de entrada e Z é o de saída) que as incorporam. O estado interno é descrito por um conjunto de estados possíveis (Q). A função de transformação do sistema é descrita por duas funções: a de transição de estado (δ) e a de saída (λ).

A função de transição de estado apresenta sempre a mesma formulação:

$$\delta : Q \times \Sigma \rightarrow Q$$

Já a função de saída apresenta duas formulações distintas:

- **Máquinas de Mealy:** onde a função de saída depende das entradas: $\lambda : Q \times \Sigma \rightarrow Z$
- **Máquinas de Moore:** onde a função de saída não depende das entradas: $\lambda : Q \rightarrow Z$

Projeto - Parte 1

É importante referir que esta primeira parte do projeto foi realizada em Java, posteriormente esta parte do trabalho foi passada para Python pelo professor, dando origem à biblioteca SAE (Simulador de Ambiente de Execução).

Para uma melhor compreensão e execução do projeto pela parte dos alunos, no material de apoio fornecido [Morgado, Luís. (2024)], foi apresentado o seguinte esquema:

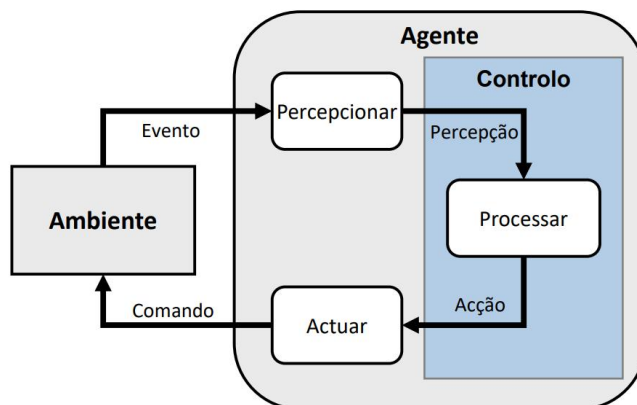


Figura 3 - Morgado, Luís. (2024). Representação conceptual da relação entre agente e ambiente

Aqui temos o problema dividido em várias partes para facilitar a abordagem, obtendo assim vários subsistemas para trabalhar. Começou-se por implementar a parte do Ambiente, que evolui ao longo do tempo, onde é possível executar comandos e observar eventos e onde é possível mostrá-los.

Com isto em mente, criou-se o package `ambiente` e nele foram desenvolvidas três Interfaces para representar cada componente do subsistema e cada Interface tem os métodos necessários para abordagem referida acima. Ambiente tem os métodos `evoluir()`, `observar()` e `executar()`, enquanto as Interfaces que representam o Comando e o Evento têm o respetivo método `mostrar()`.

Desta forma a modulação presente é baseada no encapsulamento, isolando estes detalhes internos nas interfaces para futuramente, ao ser criado um objeto que seja um Ambiente, um Evento ou um Comando, ser estabelecido um contrato funcional, o que quer dizer que as suas características (as da interface) não são diretamente implementadas. Assim também reduzimos uma possível futura complexidade que é desnecessária.

Num novo package `agente`, implementou-se as mecânicas necessárias para a reprodução do comportamento do Agente. Neste package temos as classes `Percepcao` e `Accao` e Interface `Controlo` e, por fim, a classe `Agente`, replicando assim os componentes do esquema da Figura 1. A Interface `Controlo` possui apenas o método `processar()` que recebe uma percepção. A Classe `Percepção` serve para receber os eventos observados no ambiente, enquanto a Classe `Accao` possui o comando da ação a ser executado em resposta ao ambiente.

A classe `Agente`, recebe um `Ambiente` e um `Controlo` no seu construtor e tem como métodos:

- **`executar()`** : que implementa toda a mecânica do Agente. Começando por perceber os eventos do ambiente, depois processa-os e acaba por executar a ação desejada.

- **percepcionar()** : Método retorna o que o agente vê no ambiente. Observa o ambiente e cria um objeto Percepção a partir do evento que resultou da observação.
- **actuar()** : Representa a execução da ação tomada pelo agente.

Tendo feito esta primeira abordagem com o auxílio de algumas técnicas de engenharia de software como a coesão (o seu aumento no caso), fatorização e o encapsulamento, criou-se um novo package chamado `jogo`. Neste package utilizamos as classes e interfaces já criadas para especificar os conceitos ao contexto específico do trabalho. Dentro do package `jogo` criou-se uma classe `Jogo` e mais dois packages: `ambiente` e `personagem`.

O ambiente tem uma classe chamada `AmbienteJogo` e dois enum (grupo de constantes) chamados `ComandoJogo` e `EventoJogo`.

`EventoJogo` é composta pelos vários eventos que podem ser observados no ambiente, sendo eles: Silêncio, Ruído, Animal, Fuga, Fotografia, Terminar. Já o `ComandoJogo` tem os vários comandos a serem executados: Procurar, Aproximar, Observar, Fotografar.

A `AmbienteJogo` implementa a interface `Ambiente`. No seu construtor foi implementado um dicionário baseado em `HashMap` para relacionar a tecla introduzida pelo utilizador ao um evento dos já criados em `EventoJogo`. A escolha de implementar um dicionário deu-se com a finalidade de aumentar a simplicidade do projeto, ficando mais fácil e intuitiva a sua leitura e entendimento.

Key	s	r	a	f	o	t
Evento	SILENCIO	RUIDO	ANIMAL	FUGA	FOTOGRAFIA	TERMINAR

Tabela 1 - Tabela com os valores do dicionário de `AmbienteJogo`

Como temos poucos comandos a escolha de um dicionário é justificada. Contudo, se existissem muitos comandos seria mais prudente optar por outra estrutura para substituir o dicionário, porque a cada nova combinação (com o dicionário em `HashMap`) é preciso adicionar uma nova linha.

No `AmbienteJogo` temos os seguintes métodos:

- **evoluir()** : Atualiza o evento jogo para aquele que o utilizador escolher recorrendo a `gerarEventos()`.
- **observar()** : Mostra e retorna o evento observado no ambiente, se o evento não for `null`.
- **executar()** : Executa o comando que lhe é passado. Neste caso a execução do comando é dada por um `print` na consola. É utilizado `@Override` para garantir que o método utilizado é o da interface `Ambiente`. Desta forma, se trocarmos o nome do método na interface, receberemos uma mensagem de erro.
- **gerarEventos()** : Pede ao utilizador para introduzir a letra do evento desejado e, recorrendo ao dicionário, é retornado o respetivo evento.

Por fim, no package `personagem`, temos duas classes: `Personagem` e `ControloPersonagem`.

A classe `Personagem` deriva da classe `Agente` (ou seja é uma especialização dela) e ao chamar o

construtor da classe `Agente` com a ajuda do `super()`, temos de instanciar um ambiente e um controlo para o personagem com um objeto da classe `ControloPersonagem`, porque o construtor do agente assim o exige.

O facto da classe `Personagem` derivar de `Agente` cria uma relação de acoplamento alto, mas neste caso é justificada a escolha, pois a `Personagem` é um `Agente`.

O `ControloPersonagem` torna possível, como o nome sugere, controlar a personagem. Esta classe implementa a interface `Controlo`, o que mais uma vez evidencia a importância da modularização com a ajuda do encapsulamento e fatorização, pois caso seja preciso outro tipo de controlo podemos recorrer à mesma interface.

O construtor desta classe define as ações, transições (que são definidas usando a técnica de encadeamento de método (*Method Chaining*) para facilitar a leitura e compreensão do código) e uma máquina de estados, com o seu estado inicial definido para `procura`. A implementação e respetiva explicação será abordada mais à frente.

- O `ControloPersonagem` tem um método importante chamado `processar()`, que recebe uma percepção e através dela obtemos um evento. Este evento é processado pela máquina de estados que retorna a ação associada ao evento obtido no começo e esta ação também é retornada pelo `processar()`.

Para a classe `Jogo` implementa um jogo que consiste num ambiente onde a personagem tem por objetivo registar a presença de animais ao fotografá-los.

Ao implementá-lo, optou-se por uma abordagem de decomposição, onde se dividiu o problema em `Ambiente` e `Personagem`. Com a implementação realizada no agente, ambiente e no package `jogo`, podemos tirar proveito do nível da abstração feito e de algumas especializações (como é caso do `Personagem` para com o `Agente`) evitando código repetido (por exemplo, criar um ambiente próprio para o jogo de raiz) e, consequentemente, a redundância.

O `Jogo` tem um método `executar()` que possui um *loop* para responsável por evoluir o ambiente e garantir que o personagem execute as suas ações. Este *loop* dura até o evento que o jogador passar não for `TERMINAR`.

Dentro da pasta `iasa_jogo`, além dos packages `agente`, `ambiente` e `jogo`, que já foram mencionados, também temos o package `maquest` que implementa os mecanismo necessários para a criação de uma máquina de estado.

A máquina de estados seguida para a implementação da dinâmica do personagem é a seguinte:

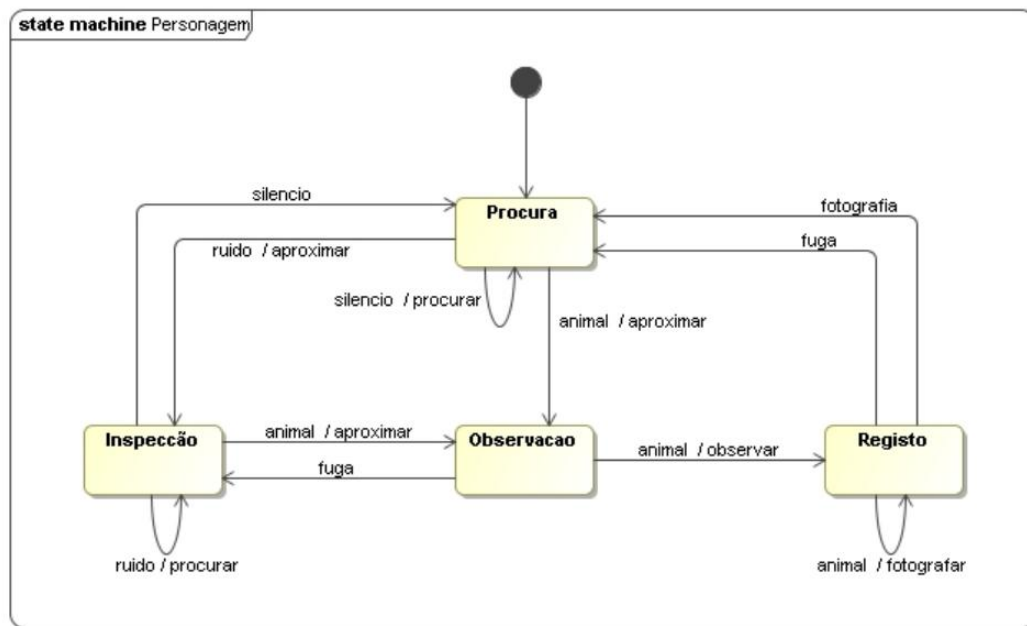


Figura 4 - Morgado, Luís. (2024). Dinâmica da Personagem

A leitura a ser feita com esta máquina de estados é: cada retângulo representa um estado, sendo que aquele que é precedido por uma bola preta é o estado inicial. As setas de transição têm sobre elas um evento (que ao decorrer, permite a transição) e, nalguns casos, é seguido por uma ação.

Dentro deste package temos três classes: Estado, Transição e MáquinaEstados, decompondo assim a máquina de estados nas partes que a constituem.

A classe Estado representa um Estado de uma máquina de estados. Um Estado, neste contexto, representa uma situação de evolução a que um objeto no sistema está sujeito. No seu construtor, é recebido uma string que representa o nome do estado e é inicializado um dicionário (baseado em HashMap) que relaciona o Evento com a Transição, implementando as tabelas das funções lambda e sigma.

Além do construtor temos os métodos:

processar(): Acedemos ao dicionário das transições através do evento que o método recebe para obter a Transição associada.

transicao(): Criou-se dois métodos com este nome, porque podem existir transições que não apresentam uma ação, ou seja aplicamos a função sigma por não termos qualquer ação associada à expressão. Para contornar esta situação, recorreremos à fatorização, onde temos uma função que recebe um evento, um estadoSucessor e uma accao (por termos uma ação aplicamos) e então inserimos uma nova entrada no dicionário e retornados o próprio objeto para possibilitar o encadeamento referido no ControloPersonagem. Depois temos uma outra função que recebe apenas um evento e um estadoSucessor e tornamos um objeto da primeira função, mas com a ação a null.

A classe Transicao representa uma transição de uma máquina de estado. Uma Transição é um acontecimento que faz com que o sistema evolua de estado.

Nesta classe, excluindo funções para *get* temos o construtor que recebe um estadoSucessor e uma

acao.

A classe `MaquinaEstados` recebe no seu construtor o estado inicial da máquina e tem um método `processar()`, que ao receber um evento e processá-lo, atualizamos o estado da máquina.

A primeira parte do projeto foi finalizada neste ponto e abaixo é apresentado um *print* do resultado do jogo:

```
PS C:\Users\miguel\Desktop\iasa50746> & 'C:\Program Files\Java\jdk-19\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionStack' -jar 'C:\Users\miguel\Desktop\iasa50746\redhat.java\jdt_ws\iasa50746_99eebab2\bin\iasa_jogo.src.jogo.Jogo'

Qual é o evento?
o
Percepção FOTOGRAFIA
Estado: Procura

Qual é o evento?
s
Percepção SILENCIO
Estado: Procura
Acção: PROCURAR

Qual é o evento?
f
Percepção FUGA
Estado: Procura

Qual é o evento?
r
Percepção RUÍDO
Estado: Inspeção
Acção: APROXIMAR

Qual é o evento?
t
Percepção TERMINAR
Estado: Inspeção
PS C:\Users\miguel\Desktop\iasa50746>
```

Figura 5 - Demonstração do jogo feito na Parte 1

Projeto - Parte 2

Esta segunda parte do projeto aprofunda a função de um agente num sistema, apresentando o conceito de agente reativo e também somos introduzidos aos princípios que sustentam a Inteligência Artificial.

O objetivo é implementar a arquitetura de um agente reativo que é definida pelo ciclo de PERCEÇÃO - PROCESSAMENTO - AÇÃO, já mencionado no [Enquadramento Teórico](#). Neste contexto do agente trabalhado na sala de aula, o ciclo é idealizado da seguinte forma:

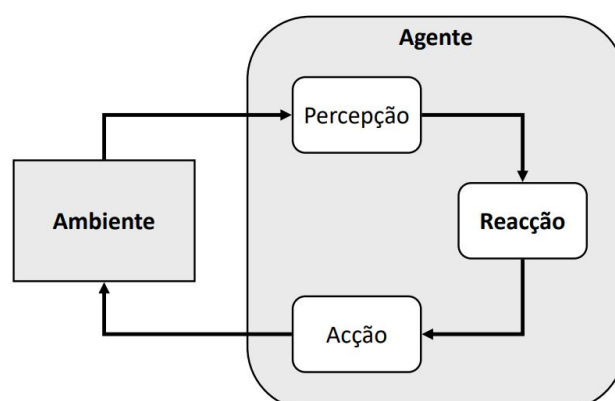


Figura 6 - Morgado, Luís. (2024). Arquitetura de um agente reativo

Para começar a implementação em código, desta vez em Python, criou-se a biblioteca `erc` (Esquemas Comportamentais Reactivos) e a primeira parte dos mecanismos de reação a ser trabalhada foram as associações estímulo-resposta.

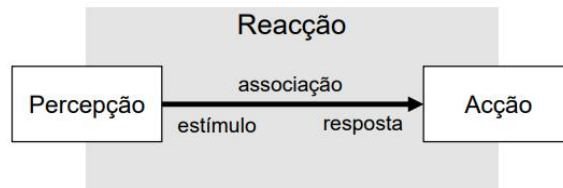


Figura 7 - Morgado, Luís. (2024). Associação Estímulo-Resposta

A Ação é diretamente ativada em função da Percepção, onde a Percepção origina um estímulo e a resposta a este estímulo origina uma Ação. Associação existente entre o estímulo e a resposta é designada por Reação.

Começou-se por fazer o código do Modelo de estrutura que consiste no Estimulo, na Resposta e na Reação.

Um Estímulo consoante a percepção que recebe, deteta uma determinada intensidade para a reação resultante. Para implementá-lo criou-se uma interface, que em Python é uma classe abstrata com métodos abstratos. Esta interface tem um método `detetar()` que recebe uma `percepcao` que dará origem ao estímulo para que depois este ative uma reação.

A classe `Resposta` representa uma resposta que é responsável por definir qual a ação a realizar e a sua prioridade, consoante o estímulo recebido. Recebe uma ação no seu construtor e tem uma função chamada `activar()`, que recebe uma `percepcao` e uma intensidade, que por padrão é definida como 0. Esta função tem como objetivo ativar uma resposta para retornar uma ação, atribuindo o valor da intensidade à propriedade da ação.

A classe `Reacao` implementa a interface `Comportamento`, que já será abordada. Uma reação é responsável por definir qual a ação a realizar e a prioridade desta, consoante o estímulo que recebe, criando então uma associação entre estímulo e resposta.

O construtor desta classe recebe exatamente o estímulo e a resposta e a função `activar()`, que recebe uma `percepcao`, associa os estímulos às respostas, onde o estímulo detetará uma certa intensidade, dada a percepção que recebe, e é ativada uma resposta com essa intensidade. A ativação dessa resposta resulta na ação a ser tomada, que é retorno desta função.

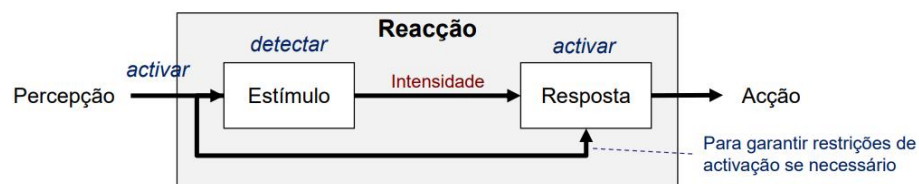


Figura 8 - Morgado, Luís. (2024). Esquema da associação Estímulo-Resposta

A Interface `Comportamento`, que é implementada pela classe `Reacao` e `ComportComp` que representa um comportamento composto, e possui uma função `activar` que recebe uma `percepcao` para ativar o respetivo comportamento. Um comportamento é um conjunto de reações (que por sua vez também são

comportamentos/reações), que consoante o resultado da percepção reproduzem uma determinada ação.

A classe que representa um Comportamento Composto e recebe no seu construtor a lista de comportamentos que compõe o comportamento composto. A função `activar()` percorre a lista de comportamentos e para cada reação coloca-se a respetiva ação na lista `accoes`, de acordo com a ação passada.

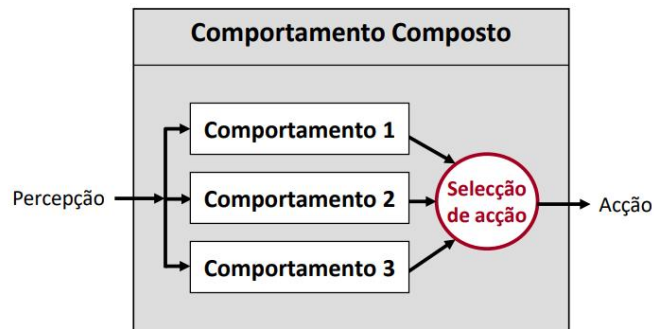


Figura 9 - Morgado, Luís. (2024). Comportamentos Compostos

O mecanismo de seleção de ação serve para determinar qual a ação a tomar, consoante a resposta dos vários comportamentos internos. Temos mecanismos de seleção baseados em Hierarquia, Prioridade e Combinação.

- **Hierarquia:** Os comportamentos são organizados numa hierarquia fixa de subsunção (suprime e substitui). Na imagem seguinte, temos este modelo de seleção exemplificado, onde a Reação 1 está no topo da hierarquia e por isso suprime e substitui as outras reações até obtermos uma ação.

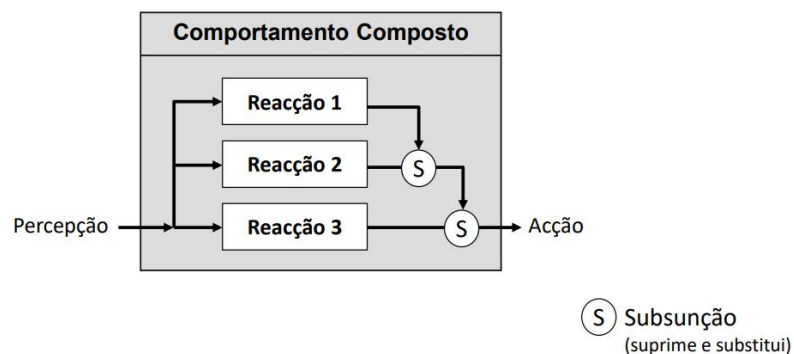


Figura 10 - Morgado, Luís. (2024). Seleção por Hierarquia

- **Prioridade:** As ações são seleccionadas consoante uma determinada prioridade que varia ao longo da execução.

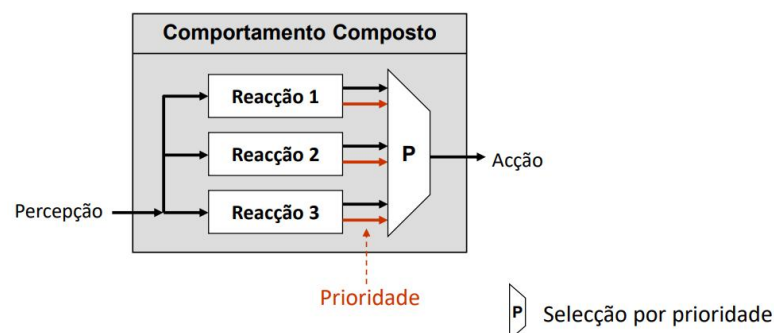


Figura 11 - Morgado, Luís. (2024). Seleção por Prioridade

- **Combinação:** As respostas são fundidas, resultando num só resposta por composição.

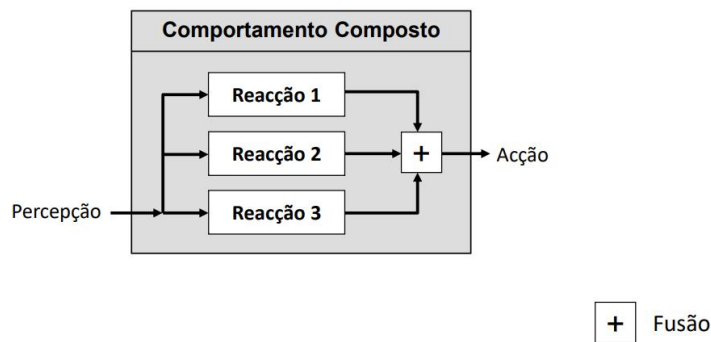


Figura 12 - Morgado, Luís. (2024). Seleção por Combinação

No código, continuou-se o trabalho com a implementação dos métodos de seleção, nomeadamente a seleção por Hierarquia e a por Prioridade. Sendo que as duas classes são uma especialização da classe `ComportComp` e têm apenas uma função chamada `selecionar_accão()`, diferenciando-se apenas pelo seu funcionamento.

A classe `Hierarquia` funciona de maneira simples: consulta a lista de ações que lhe é passada e retorna a primeira. Já a classe `Prioridade` percorre a lista de ações e, com a ajuda da função `max()` e de uma função `lambda`, retorna a ação prioritária.

A função `max()` é uma função funcional que retorna o maior valor de uma lista, enquanto a função `lambda` tem como argumentos uma ação da lista e retorna a sua prioridade. Uma função `lambda` é uma função anónima em que só precisamos de passar os argumentos e o retorno desejado ou a expressão para obtê-lo:

```
lambda argumentos : retorno
```

A estrutura implementa depois dos mecanismos de seleção de ação foi o controlo reativo do agente. Como é mostrado na Figura 1, o agente tem um módulo de controlo onde processa as perceções para depois as relacionar com uma determinada ação. Como estamos a fazer um agente reativo, o seu módulo de controlo também será reativo, onde o processar das perceções é realizado com base no módulo de Comportamento, o qual representa o comportamento geral do agente que pode ser composto por sub-comportamentos.

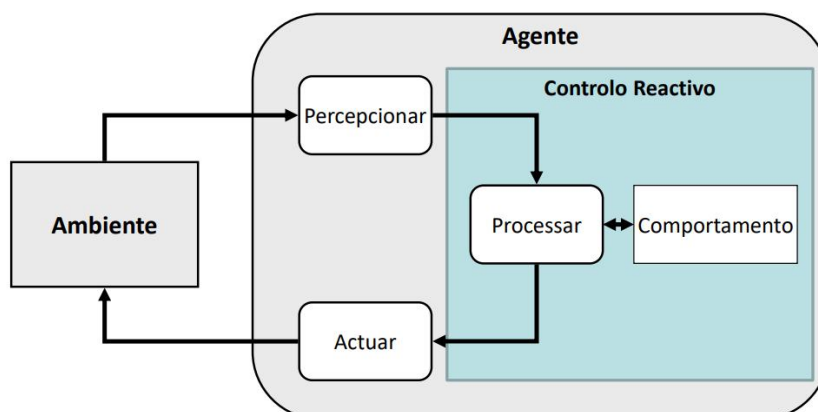


Figura 13 - Morgado, Luís. (2024). Agente com Controlo Reactivo

Criou-se então a classe `ControloReact` que é uma especialização da classe abstrata `Controlo` da biblioteca `sae`. Mais uma vez podemos tirar proveito da modularização e fatorização feita na estrutura do projeto

para diminuir a complexidade e redundância de código. Pois sendo filha da classe `Controlo`, o `ControloReact` precisa somente de especificar o seu método de processamento na função `processar()` que herda, onde através da percepção passada, ativa-se o respetivo comportamento.

Sendo a finalidade objetiva de um agente representa por um comportamento, os seus sub-comportamentos são os passos a efetuar para que o agente alcance o objetivo desejado. Assim sendo, o sistema que representa o Controlo Reativo subdivide-se em reações que, por sua vez, divide-se em Explorar e Reposta.

Desta forma, podemos implementar o comportamento Explorar, onde o agente se movimenta em direções aleatórias pelo mapa. Respeitando a estrutura do subsistema, criou-se um package `reaccoes` e dentro dele um `explorar` e outro `resposta`.

O package `resposta` possui dois tipos de resposta: um `RespostaMover` e um `RespostaMoverAleatoria`, onde o primeiro tipo é uma especialização de `Resposta` e, devido a isso, podemos chamar o construtor da classe mãe, que já recebe uma ação, para então instanciarmos uma ação a partir da direção que `RespostaMover` recebe no seu construtor. Assim obtemos uma resposta tendo em conta a direção desejada.

A `RespostaMoverAleatoria` deriva da classe anterior e chama o seu construtor, passando-lhe uma direção aleatória. Essa aleatoriedade é obtida com a ajuda da função `choice()` que retorna um elemento aleatório de uma lista, sendo que a lista neste caso é o enum `Direccao` da biblioteca `sae` que possui as quatro direções sobre as quais o agente se desloca.

O package `explorar` também possuiu dois tipos de exploração sendo eles: `Explorar` e `ExplorarRodar`, sendo que ambos implementam a interface `Comportamento`. A classe `Explorar` ao receber uma percepção para ativar o respetivo comportamento, por ser um comportamento fixo, só tem uma resposta que é mover-se aleatoriamente, assim sendo a função `activar()` retorna ação ativada pela resposta.

Na classe `ExplorarRodar` é criado um dicionário (`direccao_rodar`) que é usado para que o agente rode no sentido dos ponteiros do relógio, tendo associação de direções, em pares, para tal finalidade. A sua função `ativar()`, através da propriedade `direccao` da `percepcao` passada à função, obtém direção atual do agente. Em seguida, cria-se uma resposta do tipo `RespostaMover` que recebe a próxima direção a seguir, consultando o dicionário criado. Por fim, através desta resposta, ativa-se a respetiva ação.

O objetivo é fazer agente autónomo inteligente que seja capaz de navegar num mapa com obstáculos e alvos espalhados, desviando-se desses mesmos obstáculos e recolhendo os alvos. Por outras palavras o comportamento que representa o nosso objetivo final é Recolher, que é composto pelos sub-comportamentos `AproximarAlvo`, `EvitarObst` e `Explorar`, sendo que este último já foi implementado.

No package `reaccoes` criou-se outro chamado `aproximar`. O comportamento `AproximarAlvo` como se aproxima do alvo mais próximo, dividiu-se este problema em três classes: `EstimuloAlvo`, `AproximarDir` e então `AproximarAlvo`. Desta forma simplificamos a abordagem ao problema e conseguimos reaproveitar código já implementado.

A classe `EstimuloAlvo`, que implementa a interface `Estimulo`, retornará a quantificação da intensidade do estímulo para uma determinada distância, caso se detete um alvo. O seu construtor recebe uma direção e um

valor gama (por defeito, tem o valor igual a 0,9). A função `detetar()` que recebe uma percepção direcional e com um acesso indexado, criado na biblioteca `sae`, obtemos o elemento e distância da percepção obtida, ignorando a posição que também é retornada. Se o elemento for um alvo, quantificamos a intensidade do estímulo a partir da distância e retornamos esse valor; caso contrário o valor retornado é zero.

A quantificação da intensidade é feita tendo em vista que num intervalo de valores entre 0 e 0.9, quando elevados a um valor n , o resultado tende a decrescer. Assim se n for a distância, ficamos com um decaimento inverso à distância, ou seja, quanto maior for a distância, menor será a respetiva intensidade.

`AproximarDir` recebe uma direção no seu construtor e como esta classe deriva da `Reacao`, que tem como argumentos do construtor um estímulo e uma resposta, podemos aproveitar esta relação para chamar o construtor da classe mãe através do `super()` e passamos um `EstimuloAlvo` e uma `RespostaMover` com a direção de `AproximarDir` associada.

Por fim, a `AproximarAlvo` é um comportamento que se aproximará da direção que tiver maior prioridade e, por este motivo, deriva da classe `Prioridade`. No seu construtor cria-se uma lista `accoes_direcao` a ser retornada com as ações associadas às respetivas direções por ordem de prioridade.

Percorre-se a lista `enum Direccao` da `sae`, que contem as quatro direções sobre as quais o agente pode andar, e a partir de cada direção, adiciona-se a respetiva ação à lista `accoes_direcao`, com a ajuda de `AproximarDir`. Por último, chama-se o construtor da classe mãe que fará a gestão de qual direção têm maior prioridade.

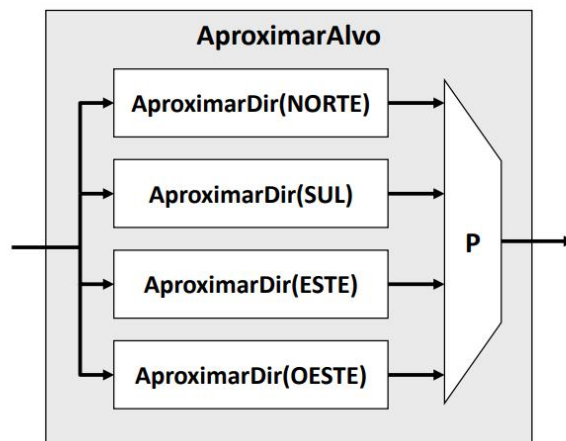


Figura 14 - Morgado, Luís. (2024). Comportamento *AproximarAlvo*

Para o comportamento evitar optou-se por uma abordagem bastante semelhante. Criou-se um package `evitar` dentro do `reacoes` e criaram as seguintes classes: `EstimuloObst`, `RespostaEvitar`, `EvitarDir`, `EvitarObst`.

`EstimuloObst` implementa a interface `Estimulo` e recebe uma direção e uma intensidade, por defeito com o valor 1, no seu construtor. A classe apresenta o método `detetar()`, que recebe uma percepção e através da sua função `contacto_obst()`, criada na `sae`, verificamos se existe algum contacto com um `Elemento` que seja um obstáculo. Caso seja verificada a existência de contacto, retorna-se a intensidade, caso contrário é retornado 0.

A `RespostaEvitar` é um tipo de resposta que será responsável por dar memória ao agente para que este saiba evitar obstáculos. O seu construtor recebe a direção inicial do agente (neste caso foi definido que seria Este) e chama-se o construtor da super classe, neste caso é a classe `RespostaMover`, e define-se uma lista com as quatro direções possíveis.

A função `direccao_livre` é um método privado que recebe uma perceção e verifica para coloca todas as direções que não apresentam um obstáculo numa lista de direções livres e retorna uma dessas direções livres, de forma aleatória. A função `activar()` recebe uma perceção e um valor de intensidade e começa por verificar se a direção atual apresenta alguma colisão com um obstáculo, caso seja verdade, é atribuída uma direção livre e se esta for válida atualizamos a direção e ativa-se a resposta nesse sentido.

Para facilitar a compreensão do algoritmo da função `activar()`, abaixo está demonstrado o esquema seguido para a sua realização:

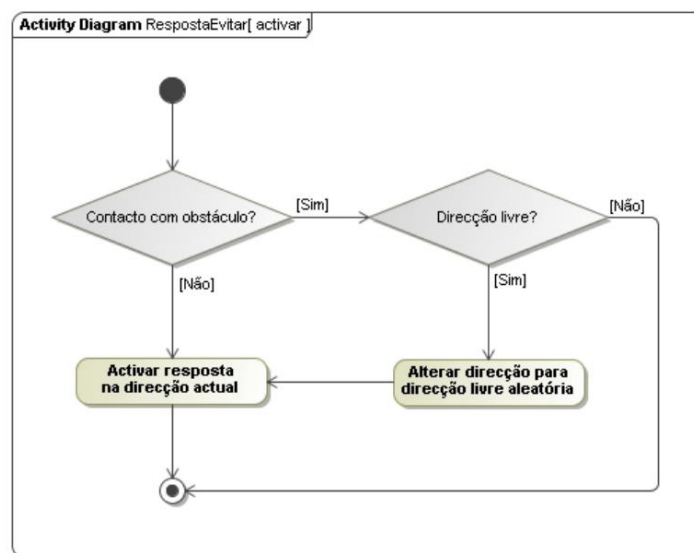


Figura 15 - Morgado, Luís. (2024). Resposta a Estímulos

A classe `EvitarDir` deriva de `Reacao` e chama o seu construtor passando-lhe um estímulo e uma resposta, sendo que o estímulo é passado ao instanciar um `EstimuloObst` com a direção que `EvitarDir` recebe no próprio construtor.

A classe `EvitarObst` tira proveito destas implementações anteriores, derivando de `Hierarquia` e ao criar uma resposta do tipo `RespostaEvitar`. No fim, chama o construtor da classe mãe e passa-lhe uma lista de ações composta por uma reação `EvitarDir`, à qual se passa um direção e a resposta criada anteriormente, onde cada direção corresponde a uma das existentes no enum `Direccao`.

Agora que os os sub-comportamentos estão concluídos, a realização do código do comportamento `Recolher` é bastante facilidade. Como `Recolher` seleciona os sub-comportamentos consoante uma hierarquia, a classe `Recolher` deriva de `Hierarquia` e chama o seu construtor passando-lhe a lista de comportamentos devidamente organizados.

COMPORTAMENTO RECOLHER

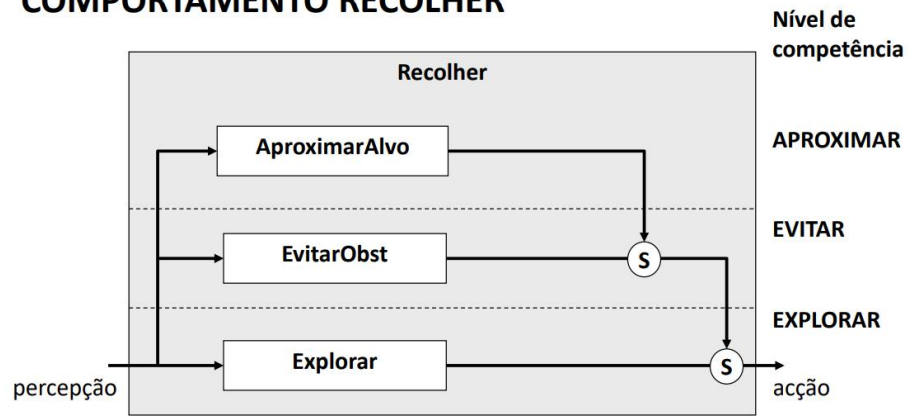


Figura 16 - Morgado, Luís. (2024) - Comportamento Recolher

Finalizado este ponto, encerra-se aqui a parte 2 do projeto. Abaixo é apresentado um *print* do resultado do jogo com o agente no mapa do lado esquerdo e o respetivo mapeamento sensorial do agente do lado direito.

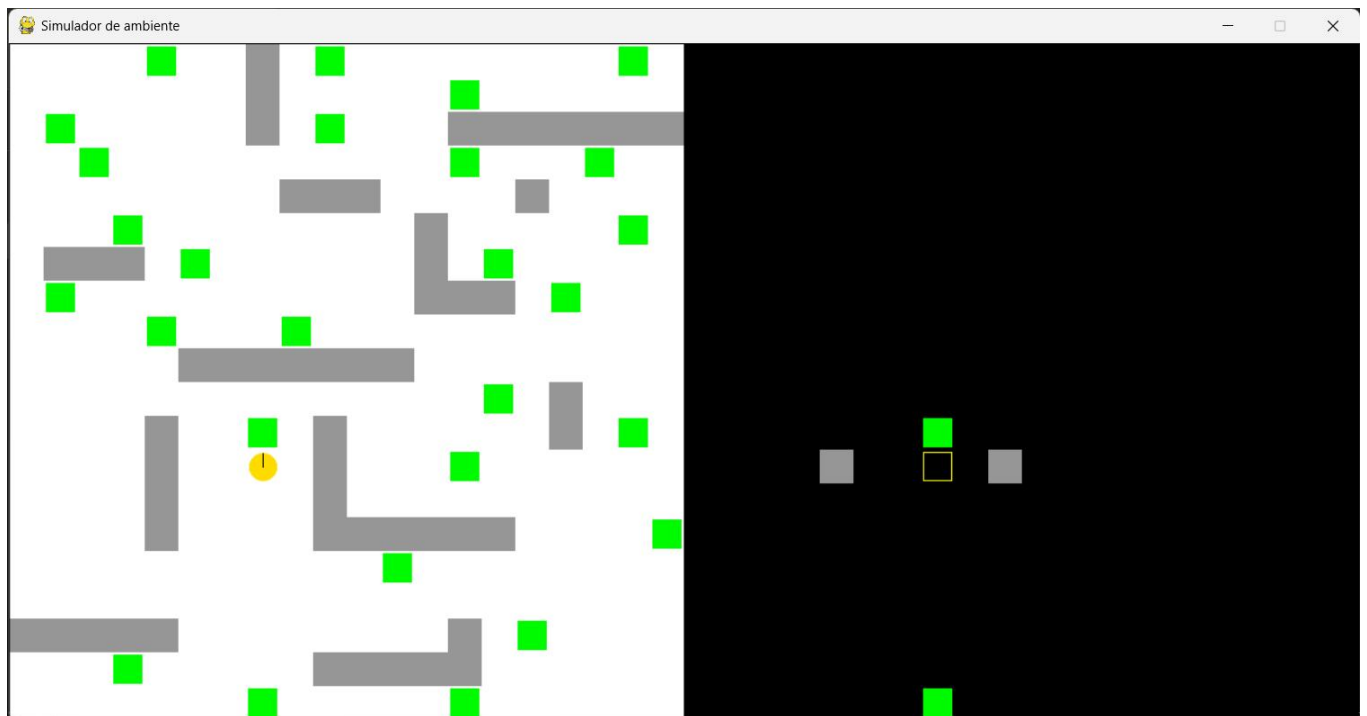


Figura 17 - Apresentação do resultado do projeto no final da Parte 2

Revisão do projeto realizado

Durante o decorrer das aulas, muito do código implementado foi explicado e demonstrado pelo professor, o que deixava pouco espaço para erros. Contudo, devido a alguma distração, entre outros fatores, algumas entregas foram com alguns erros.

Muitas das vezes os erros eram de sintaxe. Por exemplo, no dia da entrega do controlo reativo, o seu construtor não tinha um *undersocre* que deveria estar junto ao *init* que identifica o construtor.

Por vezes, os *imports* vindos da biblioteca *sae* tinham mais elementos do que deviam. Como a biblioteca já foi preparada pelo professor para facilitar os *imports*, apenas deveria existir *imports* do tipo *from sae import . . .*, o que nem sempre aconteceu.

A visibilidade dos atributos também sofria pela distração já que o que diferencia um objetivo ou função protegida de uma privada é um *undersocre* e, por vezes, só de uma entrega para a outra é que estes pormenores eram notados e corrigidos.

Conclusão

Nestas duas primeiras partes do projeto, acabámos por implementar um agente reativo que se desloca aleatoriamente num mapa, evitando obstáculos e recolhendo alvos enquanto o faz. Para este fim, foi necessário, em termos teóricos, perceber bem os fundamentos por detrás dos mecanismos de reação, seleção de ação e de coordenação de comportamentos que possibilitam o bom funcionamento do agente.

Ao longo do projeto foi possível compreender alguns dos princípios que sustentam a Inteligência Artificial, que é uma área da informática que, mais do que nunca, cresce e desenvolve-se a um grande ritmo.

Também se viu a importância de uma boa estrutura de código, recorrendo a boas práticas de Engenharia de Software, que facilita o trabalho do programador em qualquer etapa do projeto. A boa documentação também se demonstrou fundamental, porque mesmo compreendendo a teoria que dá suporte ao código, por vezes, não nos lembramos do porquê de determinadas opções terem sido tomadas ou de como algum mecanismo funciona, e é nestes momentos que a documentação revela-se uma grande ajuda.

Se num trabalho onde apenas um aluno mexe no código e, portanto, tem domínio de tudo o que está ali implementado, às vezes não percebe algo no projeto, é indiscutível que uma boa documentação, clara e precisa, ajuda uma equipa de programadores na sua produtividade para alcançar um objetivo comum.

Bibliografia

Consulta e Material de Apoio:

Morgado, Luís. (2024). IASA - Moodle

Livros:

Russel, S., & Norvig, P. (2021). Artificial intelligence: a Modern approach. (4th ed.). Prentice Hall.

Miles, R., & Hamilton, K. (2006). Learning UML 2.0. O'reilly.

Pressman, R. S. (2010). Software Engineering: A Practitioner's Approach. McGraw-Hill Science, Engineering & Mathematics.